

When to defer in Resource Constraint Setting

Akshat- 210050009, Anshul- 210050015

March - April, 2025

1 Introduction

In many real-world applications, deploying AI systems involves more than just building accurate models — it also requires knowing when to rely on them and when to shift to human feedback. However human effort is limited, and also each case requires a different amount of effort. This paper explores this resource allocation problem. We have accuracies for the model and for the human; the human has different accuracies at different levels of effort, the total effort is constrained, and we want to allocate the optimal effort allocation.

2 Related Works

2.1 When to show a Suggestion: GitHub Copilot [1]

2.1.1 Introduction

AI-powered code-recommendation systems like Copilot and CodeWhisperer provide code suggestions inside the programmer’s IDE with the aim of improving productivity. This paper illustrates a mechanism to learn from the user’s past acceptance and rejection decisions, whether to show a suggestion or not.

2.1.2 Ideal Optimisation objective

$$\begin{aligned}\mathbb{E}[S_{\text{shown}} \mid X, \phi] = & \mathbb{E}[\text{verification} \mid X, S, \phi] \\ & + \mathbb{P}(A = \text{accept} \mid X, S, \phi) \cdot \mathbb{E}[\text{editing} \mid X, S, \phi, A = \text{accept}] \\ & + \mathbb{P}(A = \text{reject} \mid X, S, \phi) \cdot \mathbb{E}[\text{writing} \mid X, S, \phi, A = \text{reject}]\end{aligned}$$

Here S_{shown} is the time taken when a suggestion is shown, the first term in the sum is the time taken to verify the suggestion, the second term is the time taken to edit the suggestion in case the suggestion is accepted and the third term is the time taken for the programmer to write the code by himself in case the suggestion is rejected.

We can define the suggestion utility as

$$\delta = \underbrace{\mathbb{E}[\text{writing} \mid X, \phi]}_{\text{S not shown}} - \underbrace{\mathbb{E}[S_{\text{shown}} \mid X, \phi]}_{\text{S shown}} - \underbrace{\mathbb{E}[\tau \mid X, \phi]}_{\text{latency}}$$

Here τ is the time taken by the recommendation system to generate the suggestion, ideally we should generate a suggestion if this suggestion utility is positive, but in practice learning models for predicting the editing and verification times is very complex and challenging.

However, under the assumption that the time taken by the programmer to write code on his own is less than the time taken by him to verify and edit the recommendations system's output, - if the acceptance probability is greater than a threshold, the suggestion utility defined above will be positive and it is optimal to show the suggestion.

2.1.3 CDHF: Conditional Suggestion Display from Human Feedback

The paper's proposed approach is as follows: Each time the programmer pauses typing, we decide using a predictor whether to show a suggestion. Crucially, we do this using a two-stage scheme to avoid generating suggestions when we know the programmer would reject them

$$m(X, S) = r(X) \cdot m_1(X) + (1 - r(X)) \cdot m_2(X, S)$$

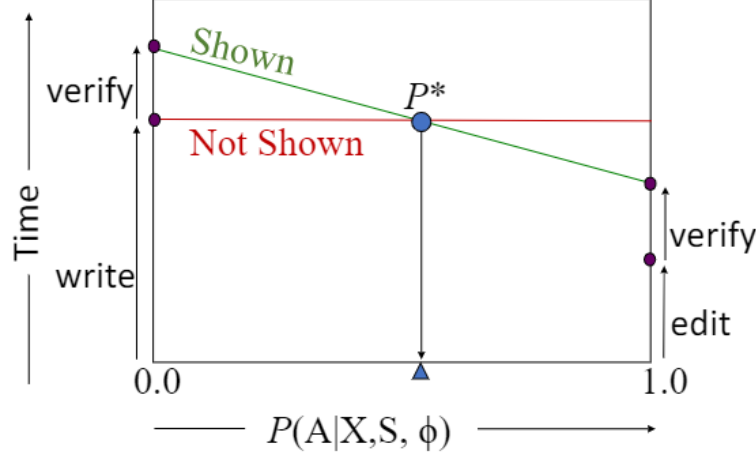


Figure 1: On the x axis we have acceptance probability and on the y axis we have time taken by the programmer, we can see that if the acceptance probability is greater than P^* the green line which denotes the time taken when a suggestion is shown is below the red line

Here, X is some representation of the code at this stage,

Display Decision. Let $m(X, S)$ be a binary predictor that denotes whether, at a given moment in the code X , we should show the suggestion S ; we call this the display decision. If $m(X, S) = 1$, we display the suggestion; otherwise, we do not. The most straightforward way to build such a function m is to estimate the programmer's probability of accepting the suggestion: $\mathbb{P}(A = \text{accept} \mid X, S)$ and then threshold the probability so that suggestions that fall below a probability t are hidden. *However, this will lead us to generate suggestions including those that will never be shown*, thus wasting computing resources. We propose to decompose the function m so that we first decide using only the code whether we can make the display decision without generating the suggestion S with a function $r(X)$. If $r(X) = 1$, we make the display decision using a stage 1 model $m_1(X)$ without generating the suggestion, otherwise if $r(X) = 0$ we generate the suggestion S and make the display decision with a stage 2 model $m_2(X, S)$ as follows:

2.1.4 Training

$$\max_{r, m_1, m_2} \quad \lambda \mathbb{E}[1 - m(X, S)] + (1 - \lambda) \mathbb{E}[r(X)] \quad (5)$$

$$\text{s.t.} \quad \mathbb{P}(A = \text{reject} \mid m(X, S) = 0) \geq p \quad (6)$$

Parameterization. We first learn calibrated estimators of the probability of accepting suggestions $\hat{\mathbb{P}}(A = \text{accept} \mid X, S)$ (with suggestion) and $\hat{\mathbb{P}}(A = \text{accept} \mid X)$ (without suggestion). We then parameterize:

$$m_1(X) = \mathbb{I}_{[\hat{\mathbb{P}}(A=\text{accept}|X) \geq t_1]}, \quad m_2(X) = \mathbb{I}_{[\hat{\mathbb{P}}(A=\text{accept}|X, S) \geq t_2]}$$

and $r(X) = \mathbb{I}_{[H(\hat{\mathbb{P}}(A=\text{accept}|X)) \leq t_4]}$ ($H(\cdot)$ is Shannon's Entropy), and optimize *jointly* over the tuple of thresholds t_1, t_2, t_4 over $[0, 1]^3$.

2.1.5 Results

o illustrate what CDHF can accomplish, we can hide 25.3that 94.7% of them would have been rejected and avoid generating 12.9% of the suggestions. If we have no concerns for latency, we can hide 52.9% of suggestions while guaranteeing that 91.3% of them would have been rejected.

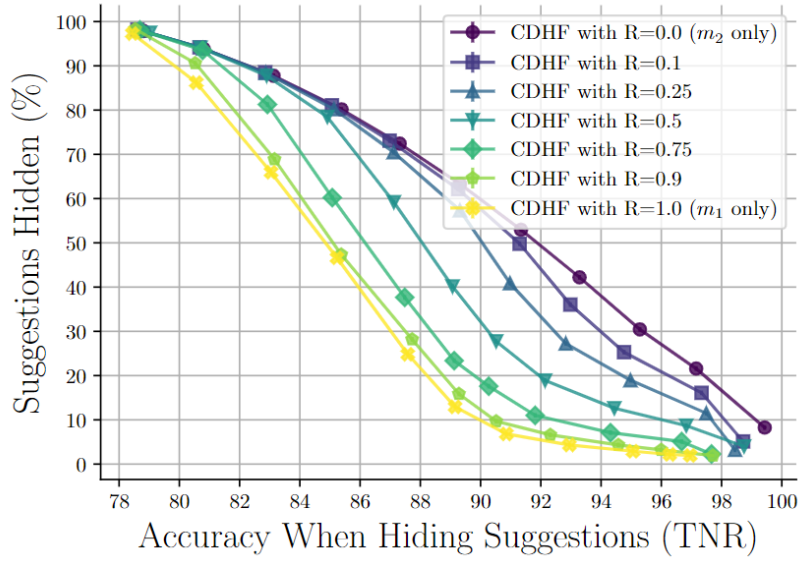


Figure 2: R here controls the degree to which models m_1 or m_2 were used, we can see that we can hide more suggestions for the same TNR if we use m_2 but using m_2 requires generating suggestions and this plot doesn't factor in for that

2.2 The Algorithmic Automation Problem: Prediction, Triage and Human Effort [2]

2.3 Introduction

As algorithms increasingly match human experts, automation is often framed purely as a performance comparison. However, this paper argues that effective automation also involves deciding which instances to assign to algorithms versus humans. It introduces a framework that formulates this allocation as an optimization problem and demonstrates that simple heuristics can improve outcomes in domains like medical AI. The findings emphasize the importance of estimating human and algorithmic errors on a per-instance basis to enhance automation effectiveness.

The automation problem should involve an algorithm that on any given instance,

1. Produces a prediction output
2. Produces a triage judgment of its effectiveness relative to the human effort it would replace in that instance.

2.3.1 Problem Formulation

The general problem can be framed as selecting a subset S of instances to be handled solely by the algorithm, with no human effort applied. The remaining instances, denoted $T = U \setminus S$, will receive human attention. Given a total budget B for human effort, an allocation of k_x units of effort is assigned to each instance $x \in T$. For each such instance x , the resulting loss is $f(x, k_x)$, while for instances $x \in S$, the loss incurred from algorithmic prediction is $g(x)$.

The final optimization objective then is as follows:

$$\begin{aligned} \text{Min} \quad & \sum_{x \in S} g(x) + \sum_{x \in T} f(x, k_x) \\ \text{subject to} \quad & \sum_{x \in T} k_x \leq B \\ & S \cup T = U; \quad S \cap T = \emptyset \end{aligned}$$

Given the estimates for the functions f and g , one can obtain further performance improvements purely through better allocations of human effort in the optimization problem.

2.3.2 Heuristics

The paper introduces a framework for optimizing the allocation of tasks between algorithms and human experts. The key heuristics are:

1. **Algorithmic uncertainty estimation:** Utilize the variance of the algorithm’s predicted probability, $m(x)(1 - m(x))$, as a measure of uncertainty for each instance x thereby, guiding the human effort allocation.
2. **Error Prediction:** Estimate separate error models for algorithmic loss $g(x)$ and human loss $f(x, k_x)$ for a given instance.
3. **Uniform Human Effort:**

Restrict attention to solutions in which each instance in T receives the same amount of effort. Suppose there are N total instances in U , and let $\alpha \in [0, 1]$ be a real number. Perform full automation on a set S of αN instances, and divide the B units of human effort evenly across the remaining $\beta = 1 - \alpha$ fraction of the instances. Then each instance in the set receiving human effort gets $B/\beta N$ units of effort. Assuming $B = cN$, the human effort per instance in this remaining set becomes c/β . With this allocation of effort, the resulting loss is:

$$\sum_{x \in S} g(x) + \sum_{x \in T} f(x, c/\beta)$$

4. **Allocation:** For a given value of α , it is sufficient to rank all instances $x \in U$ by

$$\tau_\alpha(x) = f(x, c/\beta) - g(x),$$

and then select the αN instances with the largest values of τ_α to include in the set S assigned to the algorithm. The function $\tau_\alpha(x)$ serves as the *triage score* of instance x , representing the effect of algorithmic triage relative to human effort on the expected error.

2.3.3 Results

The paper uses diabetic retinopathy data to classify cases as referable or non-referable to compare the results of their Heuristics with two baselines:

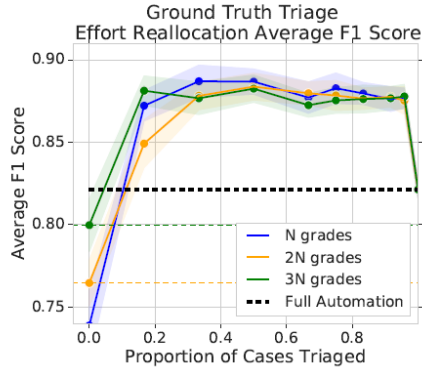
- The first is *full automation*, in which the overall loss is

$$\sum_{x_i \in U} g(x_i).$$

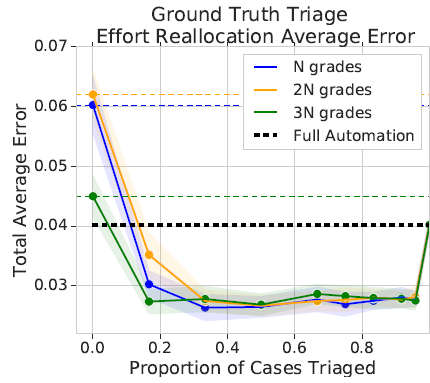
- The second is *equal coverage of all instances by human effort*. Given a budget of $B = cN$ units of effort for N instances, c units of human effort are allocated to each instance, resulting in a loss of

$$\sum_{x_i \in U} f(x_i, c).$$

The results suggest that combining algorithmic and human effort by triaging outperforms full automation and the equal coverage human baseline. Figure 3 shows the average F1 scores and Error of allocation for different values of α



(a) Avg F1 scores of allocation



(b) Avg Error rate of allocation

Figure 3: The black dotted line is the performance of full automation, and the coloured dotted lines the performance of equal coverage for the different total number of doctor grades available. Triaging and combining algorithmic and human effort perform better than all of these baselines.

3 All or Nothing Scheme ¹.

3.1 Setup

In our formulation, we work under the case where, for each instance x , an effort $e(x)$ is required corresponding to the lowest possible human loss $f_m(x)$. The loss from AI model for this instance x is $g(x)$. The error curve $f(x, e)$ ideally should be a decreasing function of the amount of effort e for a fixed instance x . The effort values can be continuous, and we take different forms of these curves to compare against the Heuristic based allocation scheme used in 2.3.2.

3.2 Formulation

We solve the problem under the case that we either put full human effort required to achieve the lowest possible achievable loss from the human model for that instance, or leave this instance to the model, so the optimisation problem becomes

$$\begin{aligned} \text{Min} \quad & \sum_{x \in S} g(x) + \sum_{x \in T} f_m(x) \\ \text{subject to} \quad & \sum_{x \in T} e(x) \leq B \\ & S \cup T = U; \quad S \cap T = \emptyset \end{aligned}$$

Say $|U| = N$, then the problem can be formulated as a Boolean linear program as follows:

$$\begin{aligned} \text{Min} \quad & \sum_{i=1}^N ((1 - V_i) \cdot g(x_i) + V_i \cdot f(x_i)) \\ \text{subject to} \quad & \sum_{i=1}^N V_i \cdot e(x_i) \leq B, \\ & V_i \in \{0, 1\} \quad \text{for all } i \in [N], \end{aligned}$$

¹The code for our implementation and experiments can be found here

Another formulation for the objective can be as follows:

$$\text{Min} \quad \sum_{i=1}^N (g(x_i) + V_i \cdot (f(x_i) - g(x_i)))$$

which simplifies to

$$\text{Min} \quad \sum_{i=1}^N V_i \cdot (f(x_i) - g(x_i))$$

3.3 Equivalence to Knapsack

3.4 0-1 knapsack

The classical 0-1 knapsack problem is given as follows:

$$\begin{aligned} & \text{Max} \quad \sum_{i=1}^n v_i x_i \\ & \text{subject to} \quad \sum_{i=1}^n w_i x_i \leq W \\ & \quad x_i \in \{0, 1\} \quad \text{for } i = 1, 2, \dots, n \end{aligned}$$

This is equivalent to our problem with
 $v_i := (g(x_i) - f(x_i)) \quad x_i := V_i \quad w_i := e(x_i).$

This problem can be solved in two ways, one using the dynamic programming algorithm for 0-1 knapsack refer, this requires the budget and effort values to be integers. Its time complexity is $\mathcal{O}(N \times W)$, which may not be feasible for a large Budget value W .

The other approach is Integer linear programming. We used PuLP library in python to solve a linear program given synthetically generated data corresponding to triplet (f_i, g_i, e_i) , where $g_i := g(x_i)$, $f_i := f(x_i)$ and $e_i := e(x_i)$. The time complexity of this approach is $\mathcal{O}(2^N)$. This may not be feasible for a large number of samples.

To have a faster solution, we can use an algorithm that gives an almost perfect approximation of the true solution, it is described below.

3.5 Fractional knapsack

A relaxed version of the problem with $0 \leq x_i \leq 1$. This is no longer an Integer linear program and in fact has a greedy solution which is given as follows:

1. Calculate the value-to-weight ratio for each item:

$$\text{ratio}_i = \frac{v_i}{w_i}$$

2. Sort all the items in descending order of their value-to-weight ratio.
3. Initialize the knapsack with a weight of 0 and a total value of 0.
4. For each item in the sorted list:
 - If the item can be fully added to the knapsack without exceeding the weight capacity, add it completely.

if $w_i \leq W$ then add the entire item.

- Otherwise, add as much of the item as possible to fill the remaining capacity of the knapsack.

if $w_i > W$ then add a fraction of the item.

5. Repeat the process until the knapsack is full or all items are considered.

The time complexity fractional knapsack is $\mathcal{O}(N \times \log(N))$.

We follow a similar greedy approach to find the allocation as described in algorithm 1. We find that the allocations obtained from using fractional knapsack and that from Integer linear program are almost similar (refer to figure 5) and hence is a good approximation.

Algorithm 1 Allocation algorithm

```
1: Initialize  $X \leftarrow \text{zeros}(N)$  {Array to store selected items (0 or 1)}
2: Calculate efficiency ratio  $\text{eff\_ratio}_i = \frac{v_i}{w_i}$  for all items
3: Sort the indices of items by  $\text{eff\_ratio}_i$  in descending order:
    $\text{sorted\_indices} = \text{argsort}(\text{eff\_ratio})$ 
4: Set  $\text{curr} \leftarrow 0$  {Current total weight}
5: Set  $\text{threshold\_er} \leftarrow 0$  {Threshold for efficiency ratio}
6: for each  $i$  in  $\text{sorted\_indices}$  do
7:   if  $\text{curr} + w_i \leq B$  and  $\text{eff\_ratio}_i > 0$  then
8:      $\text{curr} \leftarrow \text{curr} + w_i$ 
9:      $X[i] \leftarrow 1$ 
10:  else
11:     $\text{threshold\_er} \leftarrow \text{eff\_ratio}_i$  {Set the threshold efficiency ratio}
12:    break {Exit loop as further items can't be added}
13:  end if
14: end for
```

4 Results

4.1 Experimental Setup

The datapoints for tuples (f_i, g_i) are generated from a multivariate gaussian with bellow written mean and covariance

$$\text{mean} = \begin{bmatrix} 0.2 \\ 0.4 \end{bmatrix}, \quad \text{cov} = \begin{bmatrix} 0.2 & -0.1 \\ -0.1 & 0.2 \end{bmatrix}$$

These values are clipped between 0 and 1, also the negative covariance is taken to ensure complementarity between human and AI and to make sure any effort for triage is effective.

Points for e_i are sampled from a uniform distribution $\mathcal{U} \sim (0.5, 1)$.

4.2 Visualising allocations

Figure 4 shows the allocation obtained by

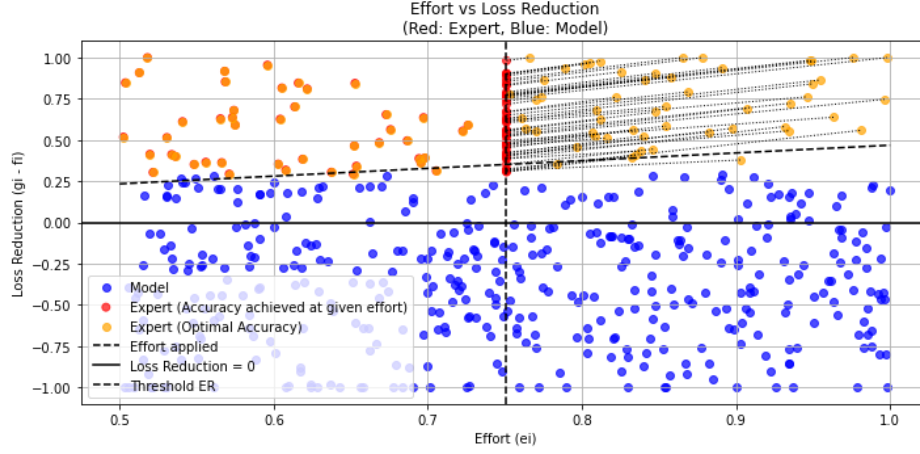
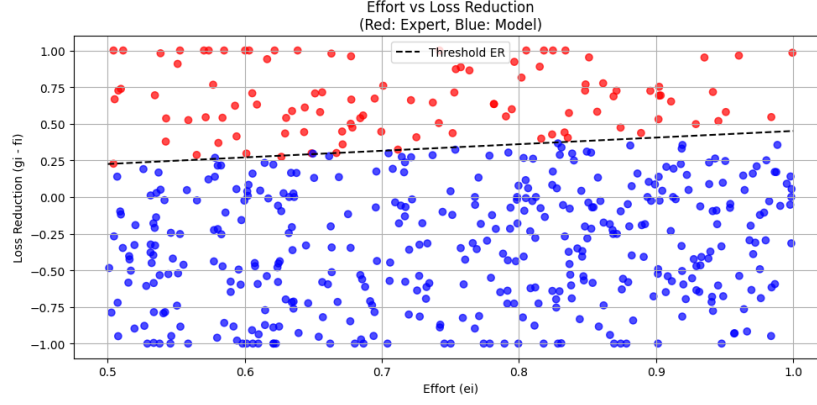
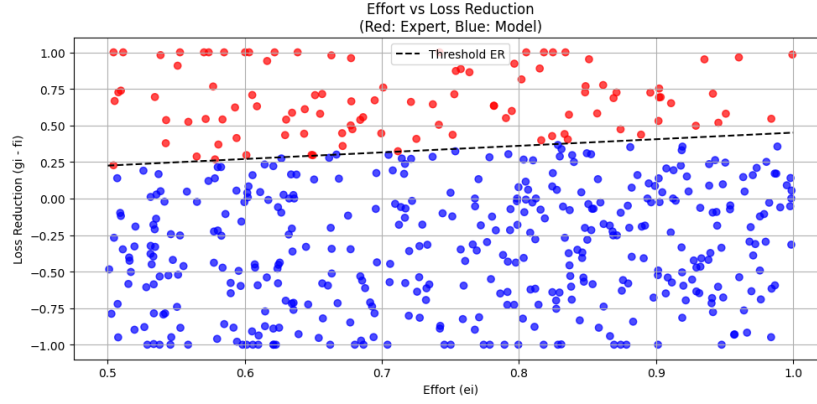


Figure 4: Allocation for the heuristics case, the dotted vertical line is the effort given, the points its left get full effort and those to its right get effort equal to that value, so accuracy is reduced for the points on the right



(a) ILP



(b) Fractional Knapsack

Figure 5: We can see that the assignment in both of the cases is approximately the same except for one or two points near the threshold efficiency ratio line, also the optimal error value obtained using each of the assignments differs only at the 5th digit of decimal, 0.157967 for ILP and 0.157989 for Fractional Knapsack

4.3 Emperical Results

We compare results for three different human error functions $f(e)$ varying with effort as shown in 6

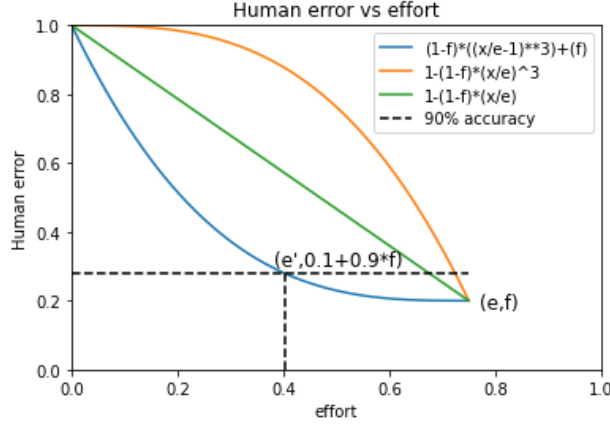


Figure 6: Error as a function of applied effort

We compare the average loss values and corresponding allocations made by the Heuristics in 2.3.2 with our allocation strategy. For simplicity, we assume that the function form for each input x_i is the same. Note that our allocation strategy doesn't care about the functional form and only cares about the optimal loss and the effort for that loss.

Table 1 shows the performance of our model as compared to heuristics approach for the three different function forms

Budget	Concave	Convex	Linear	All-or-Nothing	90% (convex)
0.01	0.2826	0.2699	0.2810	0.2812	0.2726
0.05	0.2467	0.2203	0.2456	0.2451	0.2146
0.20	0.1693	0.1489	0.1646	0.1631	0.1299
0.40	0.1315	0.1292	0.1293	0.1292	0.1299

Table 1: Comparison of average loss objective obtained by heuristics for different function forms vs All-or-Nothing allocation

4.4 Further Approach/optimization

We noticed that our approach did not beat the heuristic approach in the case when the loss variation with effort was convex. Which made sense because looking at the plot 6 for the convex case it seems that the loss plateaus a lot before the optimal effort and whole lot of effort can be saved with minimal sacrifice in accuracy.

So we fixed our allocation such that we would never give full effort to a selected instance, but would give effort that corresponds to a fraction of the optimal loss (results shown for 90%).

Note that now this approach depends on the loss function; In the table 1 we show the results of this approach when the loss function is convex, we see that this performs better than both our all-or-nothing model and the heuristics model.

However more research needs to be done in determining the optimal fraction, it seems like it should depend on the **loss-effort function** and also on the **budget allocated**.

4.5 Further Developments

- Estimating the loss curve, also each datapoint could have a different loss curve.
- Finding the optimal fraction of the full effort loss as described in the last section.

References

- [1] Hussein Mozannar et al. *When to Show a Suggestion? Integrating Human Feedback in AI-Assisted Programming*. 2024. arXiv: 2306.04930 [cs.HC]. URL: <https://arxiv.org/abs/2306.04930>.
- [2] Maithra Raghu et al. “The Algorithmic Automation Problem: Prediction, Triage, and Human Effort”. In: *CoRR* abs/1903.12220 (2019). arXiv: 1903.12220. URL: <http://arxiv.org/abs/1903.12220>.